



Game Day J6 - Opération « lundi matin » : réparer InduSense

À retenir

Format - scénario A retenu : journée J6, jeudi 03/09/2026, soit 7 h après les 35 h de modules du Sprint 3 (42 h au total), en binômes ou en solo, au choix. En scénario B strict, le Game Day vient après 42 h de modules, sur un septième jour à planifier : 49 h / 7 jours au total. Prérequis : le Sprint 3 (le dépôt à réparer est VOTRE dépôt fil rouge, saboté) · Matériel : le dépôt github.com/thomasfesq/CISIA_29062026, branche J6-gameday (URL rappelée par le formateur), Docker Desktop démarré. Livrables de fin de journée : ① le dépôt réparé - suite pytest entièrement verte avec des tests identiques à v1.0-sain, pipeline de données conforme aux chiffres de référence, API saine, stack compose vivante (API + Prometheus + Grafana) - poussé sur votre branche `reparation-<binome>` (ou votre fork) · ② un post-mortem d'une page (gabarit en fin de document) · ③ une restitution de 5 min. La règle d'or : chaque correctif est diagnostiqué (symptôme → cause racine), corrigé, prouvé (test/commande), commité (message clair). Un correctif non commité n'existe pas.

Le brief

Vendredi soir, « une petite maintenance » a été faite sur le dépôt de production InduSense. Lundi 8 h : l'installation échoue, l'API rejette les clés valides, le pipeline perd des mesures, Grafana est injoignable. Détail troublant : le prestataire jure que « les tests passaient » chez lui - et une partie de l'historique CI semble lui donner raison. Lisez `BRIEFING.md` à la racine. Vous avez la journée. Indice structurel : le tag `v1.0-sain` pointe sur le dernier état certifié conforme.

Les chiffres de référence (l'état certifié)

Ce qui doit être vrai	Valeur certifiée
<code>uv sync --extra dev</code>	s'exécute sans erreur (Python 3.13)
<code>uv run pytest -q</code>	suite complète verte (socle : loaders, temporel, API, sécurité, package · + 8 tests drift)
Jointure capteurs (<code>build_dataset</code>)	65 625 lignes, résidu $\approx 1,76$ % (merge_asof ± 90 min, par machine)
Fenêtres drift (<code>drift_windows.py</code>)	64 535 lignes · panne_v1 $\approx 5,2$ % · panne réelle $\approx 4,8$ %
Modèle drift (seuil gelé, coûts 5 000/200 €)	seuil 0,03 (théorique 0,038) · validation déc : PR-AUC 0,258 · ROC 0,817
PSI fenêtres (vs réf normale)	f1 : 0,002 · f2 : 6,834 · f3 : 0,002 · janvier : 6,203 (mais 0,001 vs réf haute)
Rappels au seuil gelé	f1 : 0,822 · f2 : 0,811 · f3 : 0,092 (concept, attendu !) · janvier : 0,868
API	<code>/health</code> → 200 · sans X-API-Key valide → 401 · rafale → 429 à partir de 60 req/min
Stack compose	API :8000 · Prometheus :9090 (2 cibles UP : <code>indusense-api</code> , <code>indusense-drift</code>) · Grafana :3000
CI GitHub	9 étapes vertes (install → lint → format → tests → build Docker → scan Trivy)

Phase 0 - Prise de poste (30 min) · « comprendre avant de toucher »

```
# le Game Day vit sur la branche J6-gameday du dépôt fil rouge :
# https://github.com/thomasfesq/CISIA_29062026/tree/J6-gameday
git clone -b J6-gameday https://github.com/thomasfesq/CISIA_29062026.git indusense-gameday
cd indusense-gameday
git switch -c reparation-<binome>          # VOTRE branche de travail (jamais sur J6-gameday direct)
git log --oneline --decorate                # que s'est-il passé, et quand ?
git tag                                    # où est l'état de confiance ?
git diff --stat v1.0-sain J6-gameday       # votre périmètre de suspicion
```

(Selon la consigne du formateur : travaillez sur votre fork du dépôt, ou poussez votre branche `reparation-<binome>` - jamais sur `main` ni `J6-gameday`.)

Lisez `BRIEFING.md` et le `README`, ouvrez votre post-mortem, démarrez la timeline.

À retenir

Tout au long de la journée, chaque phase se termine par sa définition of done (vos preuves de réussite), les compétences qu'elle travaille, et des □ questions type jury (facultatives) : ce sont les « pourquoi » que le jury pose vraiment - il évalue votre compréhension, pas votre récitation.



N'exécutez rien, ne corrigez rien encore.

- ▶ À vous demander : que touche le commit de « maintenance » (`git show --stat HEAD`) ?

Un commit de maintenance légitime modifierait-il `tests/` ? `pyproject.toml` ? la CI ?

- ▶ ☐ Bonne pratique : en incident, on gèle et on observe d'abord - un correctif à l'aveugle détruit les indices. La timeline commence à la première minute.

- ▶ À retenir : `git log` / `git show` / `git diff <tag>` = la police scientifique du dépôt ; c'est exactement pour ça qu'on exige des commits atomiques et messageés (m24).

Compétences travaillées : C9 (démarche : geler, observer, tracer) · C6 (outillage git).

Definition of done - phase 0 : vous savez lister les fichiers modifiés depuis `v1.0-sain` et vous avez classé la liste en familles (`env` / `code` / `tests` / `infra` / CI). Attention : tout ce qui a changé n'est pas forcément cassé - et deux choses au moins ne se voient pas dans un diff de contenu.

- ▶ ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : « Pourquoi geler avant de corriger - que détruit un correctif "à chaud" ? » · « À quoi sert un tag certifié (`v1.0-sain`) dans une démarche qualité ? »

Phase 1 - L'environnement (45 min)

```
uv sync --extra dev
```

Le message d'erreur est votre premier témoin - lisez-le VRAIMENT. Croisez ensuite trois sources : l'erreur, le `README.md`, le `pyproject.toml` (et leur diff vs `v1.0-sain`).

- ▶ À vous demander : quelle version de Python ce projet exige-t-il, et... existe-t-elle ?

Qui fait foi entre le `README` et le `pyproject.toml` ? Pourquoi la CI est-elle tombée dès l'étape « Install » sur le commit de maintenance ?

- ▶ ☐ Bonne pratique : réparer la source de vérité d'abord (pyproject), la doc ensuite ; un commit par correctif (`fix(env): ...` puis `docs(readme): ...`).

▶ À retenir : `requires-python` + `uv.lock` = le contrat de reproductibilité (m23) : le même interpréteur et les mêmes versions partout - poste, CI, prod. Un contrat impossible à satisfaire casse TOUT le monde d'un coup.

Compétences travaillées : C6 (intégrer et reproduire l'environnement).

Definition of done - phase 1 : `uv sync --extra dev` passe, `uv run python -c "import indusense"` aussi, 2 commits propres, timeline à jour.

- ▶ ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : « Pourquoi un `uv.lock` alors que `pyproject.toml` liste déjà les dépendances ? » · « "Le même environnement partout" : que se passe-t-il concrètement quand ce n'est pas vrai ? »

Phase 2 - Données & features : la chaîne §95 (1 h 15)

```
uv run pytest tests/test_loaders.py tests/test_temporal.py tests/test_temporal_gold.py -q
```

Plusieurs échecs - traitez-les UN par UN (symptôme → fichier → cause → correctif → re-test → commit). Tout ce dont vous avez besoin a été vu au fil rouge : les pièges des données InduSense ne sont pas des inventions, ce sont ceux du snippet 95.

▶ À vous demander (identifiants) : que doivent devenir `M-2`, `MACH_02`, `M_07` après normalisation ? Que se passe-t-il dans une jointure `by="machine"` si `MACH-2` et `MACH-02` coexistent ? (doublons cachés = mesures orphelines)

▶ À vous demander (temps) : pourquoi les timestamps de pression exigent-ils `utc=True` + suppression du fuseau ? Que vaut une tolérance de jointure de ± 5 min quand un des deux capteurs est décalé d'une heure entière de fuseau ? Comparez vos volumes aux 65 625 lignes / 1,76 % de référence.

- ▶ À vous demander (features) : dans une moyenne glissante « anti-fuite », que garantit le



`shift(1)` AVANT le `rolling()` ? Quel test le vérifie, et que raconte son message d'échec ?

► ☐ Bonne pratique : quand un chiffre de référence existe (65 625 · 1,76 %), chaque écart est un symptôme quantifié - c'est toute la valeur d'un « état certifié ».

► À retenir : normalisation d'identifiants = clé de jointure canonique (§95 Q-B1) ·

fuseaux mixtes = conversion UTC explicite · `merge_asof` tolérant = robustesse au jitter (Q-B2) ·

`shift(1)` avant `rolling` = pas de fuite du présent (m9). Quatre décisions, quatre pannes possibles.

Compétences travaillées : C3 (préparer les données : intégrité, pertinence, anti-fuite).

Definition of done - phase 2 : loaders + temporel verts, volumes conformes, ~3-4 commits.

► ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : « Pourquoi un split TEMPOREL et pas aléatoire sur InduSense ? (qu'avez-vous mesuré hier ?) » · « Pourquoi une jointure tolérante ±90 min plutôt qu'exacte ? » · « Qu'est-ce qu'une fuite de données ? Donnez les DEUX exemples InduSense. »

Phase 3 - API, sécurité, drift... et la definition of done (1 h)

```
uv run pytest -q          # la suite COMPLÈTE, cette fois
```

► À vous demander (API) : le rapport d'incident mentionne des 401 sur les appels authentifiés - information à VÉRIFIER (un rapport d'astreinte peut se tromper). Si le symptôme est là : la clé a-t-elle changé, ou ce qui la TRANSPORTE ? S'il n'y est pas : le contrat d'en-tête est-il cohérent PARTOUT - code, commentaires, tests, doc ? (comparez le nom d'en-tête attendu par l'API à celui que les

tests et la doc utilisent - qui est la source de vérité d'un contrat d'API ?)

► À vous demander (sécurité) : des 429 apparaissent au bout de... combien de requêtes ?

Une limite de débit, ça se règle par IP et par fenêtre : quelle est la politique certifiée, et laquelle est en vigueur ?

► À vous demander (drift) : un PSI de 0,18, c'est « RAS », « à surveiller » ou « fort » ?

Le module `indusense.monitoring.drift` est-il d'accord avec le cours ?

► La question qui fâche : quand tout semblera vert... vos tests prouvent-ils encore quelque chose ? `git diff v1.0-sain -- tests/` - un contrat, ça se relit. Restaurez ce qui doit l'être (`git checkout v1.0-sain -- tests/<fichier>`), puis re-testez.

► ☐ Bonne pratique : les tests se réparent par restauration depuis l'état certifié, le code par compréhension. Jamais l'inverse.

► À retenir : 401 = authentification (pas 422 : le problème n'est pas la forme) · un rate limit trop agressif est une panne de disponibilité auto-infligée · les seuils PSI 0,10/0,25 sont une convention à FIGER - inversés, une dérive forte devient « RAS » · un test affaibli avec un « TODO » doit hurler en revue.

Compétences travaillées : C6 (API) · C2 (menaces & garde-fous) · C8 (métriques de dérive) · C9 (definition of done).

Definition of done - phase 3 : `uv run pytest -q` entièrement vert, tests identiques à `v1.0-sain`

(`git diff v1.0-sain -- tests/` vide). Pause déjeuner méritée.

► ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : « Pourquoi ROC-AUC ne suffit pas ici - quand préférez-vous rappel / PR-AUC ? » · « Pourquoi un seuil de décision à 0,03 et pas 0,5 ? » · « Pourquoi 401 et pas 422 quand la clé manque ? Et 403, c'est quoi ? » · « Le rate limit protège de quelle menace, exactement (STRIDE) ? »

Phase 4 - Le pipeline drift et la salle de contrôle (1 h 15)

Rejouez le TP drift de bout en bout et comparez CHAQUE sortie aux chiffres de référence :

```
uv run python scripts/drift_windows.py
uv run python scripts/train_drift_model.py
uv run python scripts/evaluate_drift.py --fenetre 1      # puis 2, 3, janvier (+ janvier --reference haute)
```



Puis rallumez la salle de contrôle, maillon par maillon :

```
uv run python scripts/export_drift_metrics.py      # terminal 1
docker compose up -d                             # terminal 2
```

① L'API répond-elle sur le port annoncé ? (`curl localhost:8000/health` - comparez le mapping

`hôte:conteneur` du compose à ce que vous tapez) ② Prometheus (:9090, Status→Targets) voit-il ses

DEUX cibles UP - et à quelles adresses ? (un conteneur qui dit `localhost` parle de LUI-même)

③ Grafana répond-il là où on l'attend ? Le dashboard « InduSense - dérive & métriques »

se peuple-t-il quand vous relancez `evaluate_drift` ?

► À vous demander : pour chaque maillon muet, qui a raison - le script, le yml, la doc ?

Dans quel ordre diagnostiquer une chaîne producteur → collecteur → visualisation ?

► ☐ Bonne pratique : en réseau, on teste au `curl` maillon par maillon, du producteur vers le consommateur - jamais « au navigateur » directement.

► À retenir : `localhost` DANS un conteneur = le conteneur (d'où `host.docker.internal` pour joindre le poste) · syntaxe des ports = `hôte:conteneur` · le DNS interne Compose fait que `prometheus` joint l'API via `api:8000` - trois modèles de réseau différents dans une même stack.

Compétences travaillées : C7 (architecture de la stack) · C8 (mesurer & maintenir).

Definition of done - phase 4 : pipeline conforme aux chiffres (dont f3 : rappel 0,092 - cette

« anomalie »-là est NORMALE, sauriez-vous expliquer pourquoi en une phrase ?), 2 cibles UP, dashboard vivant. Capture d'écran pour le post-mortem.

► ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : «

Pourquoi Prometheus vient-il TIRER les métriques plutôt que l'API les pousse ? » · « Pourquoi p95 plutôt que latence moyenne ? » · « PSI muet + rappel effondré (fenêtre 3) : quel type de dérive, et pourquoi aucun test sur X ne peut la voir ? »

Phase 5 - Hygiène finale : la CI qui ment (30 min)

Avant de livrer, relisez l'infrastructure de confiance elle-même :

```
git diff v1.0-sain -- .github/
```

► À vous demander : que fait `|| true` collé à une commande de test ? Dans quel état la CI serait-elle après vos réparations... et qu'aurait-elle dit VENDREDI SOIR, juste après la casse ?

En quoi est-ce le sabotage le plus dangereux des quatorze ?

► ☐ Bonne pratique : la CI fait partie du périmètre de revue au même titre que le code ; une étape de test ne doit JAMAIS pouvoir échouer silencieusement.

► À retenir : une CI verte ne vaut que ce que valent ses étapes ; « qui garde les gardiens » s'applique aux tests (phase 3) ET au pipeline qui les exécute.

Compétences travaillées : C9 (amélioration continue : « qui garde les gardiens ») · C6.

Definition of done - phase 5 : workflow restauré, `git log --oneline` lisible (~12-15 commits racontant la réparation), push sur votre branche/fork - et l'onglet Actions repasse au vert POUR LES BONNES RAISONS.

► ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : « À quoi sert la CI si "les tests passent sur mon poste" ? » · « Pourquoi `tests/` et `.github/` méritent-ils une protection particulière (revue obligatoire) ? »

Phase 6 - Post-mortem et restitution (45 min + 5 min par binôme ou solo)



```
# Post-mortem - Game Day indusense-gameday · binôme / solo : ...
## Timeline (heure → événement/décision)
## Pannes traitées
| # | Symptôme observé | Cause racine | Correctif (commit) | Comment l'éviter ? |
## Ce qui nous a fait perdre du temps / gagner du temps
## Les 3 choses qu'on retient
```

► Pour la restitution : parmi les 14 pannes, lesquelles auraient été attrapées par la CI ?
par une revue de code ? par rien d'automatique (et quel processus humain manque-t-il alors) ?

Ce classement EST la leçon de la journée.

- ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : « Racontez UNE décision technique du jour et l'alternative que vous avez rejetée - et pourquoi. » · « Si le parc passe de 15 à 150 machines, qu'est-ce qui casse en premier dans votre stack ? »
- ☐ Bonne pratique (soutenance) : 3 messages max, chaque affirmation adossée à une preuve à l'écran - on ne LIT pas une slide, on la commente.

Compétences travaillées (phase 6) : C1→C9 - la restitution EST une répétition de soutenance.

Définition of done de la journée (vos preuves de réussite) :

- [] `uv run pytest -q` vert, tests ET workflow identiques à `v1.0-sain`
- [] pipeline drift conforme (65 625 → 64 535 lignes · seuil 0,03 · PSI f2 6,834 · rappel f3 0,092)
- [] API : 200 / 401 / 429 aux bons endroits · compose : 2 cibles UP, Grafana :3000 peuplé
- [] dépôt propre poussé + post-mortem + restitution

Bonus des rapides - rejouer l'incident historique « +8 °C sur MACH-02 » (45 min, chrono en main)

À retenir

Mise en situation. Votre dépôt est réparé, la salle de contrôle est vivante (2 cibles UP, dashboard peuplé). Il est « 14h07 » : vous êtes l'astreinte. La fenêtre 2 du TP drift EST cet incident - vous allez le déclencher vous-mêmes, puis le gérer comme une vraie astreinte SRE, en vous chronométrant à chaque étape.

Le déclencheur (et rien d'autre - le reste, c'est à vous de le découvrir) :

```
uv run python scripts/evaluate_drift.py --fenetre 2
```

Les 3 livrables attendus (vos preuves de réussite - compétences C8 · C9) :

1. La timeline SRE horodatée (6 à 10 lignes) : heure → événement → décision → PREUVE (une commande, un chiffre ou une capture par ligne). Elle commence à la détection (« la jauge PSI vire au rouge ») et se termine à la clôture (« jauges revenues au vert, preuve à l'appui »).
2. Le runbook drift (1 page, écrit AVANT de remédier) : symptôme → diagnostic → action → vérification → escalade. Un runbook s'écrit pendant qu'on a les mains dans l'incident, pas de mémoire une semaine après.
3. Le post-mortem éclair (3 lignes) : cause racine · impact réel sur le service · prévention.

Les questions qui doivent structurer votre diagnostic (répondez-y DANS la timeline) :

- Quelles familles de features hurlent dans la table PSI - et laquelle reste étrangement muette ? En quoi ce silence est-il une signature qui oriente le diagnostic ?
- Que disent rappel et ROC au seuil gelé ? Le modèle a-t-il besoin d'être réentraîné - et auriez-vous le DROIT de le réentraîner maintenant ? (avec quelles données ?)
- L'action corrective est-elle côté modèle ou côté physique ? Qui appelez-vous ?
- `--machine MACH-02` puis une autre machine : l'anomalie est-elle locale (un capteur) ou globale (chaîne d'acquisition) ? Qu'est-ce que ça change à l'action ?
- Comment prouvez-vous le retour à la normale ? (quelle commande simule le capteur réétalonné, et qu'attendez-vous sur le dashboard, en combien de temps ?)

Chrono cible : détection ≤ 2 min · diagnostic ≤ 15 min · runbook ≤ 15 min · remédiation prouvée ≤ 10 min.



- ▶ ☐ Questions type jury (facultatif - entraînement soutenance : répondez en 30 s, à voix haute, sans notes) : « Pourquoi ne PAS réentraîner le modèle pendant l'incident capteur ? » · « Quelle dérive ne verrez-vous JAMAIS sans labels - et quel signal précoce proposez-vous à la place ? » ☐ Bonne pratique : pendant tout l'incident, le service continue de répondre - la première décision d'astreinte est toujours « faut-il couper ? » : justifiez la vôtre.